MODULE 30 · EXAM STRATEGY - ALL DOMAINS

How DVA-C02 Questions Are Written

The anatomy of a question, the six kinds of wrong answer, and the clause that decides.

TYPE Exam technique MODULE 30 of 34 SCORED 50 of 65

admin@sthithapragna.com

A hard question is rarely testing a hard fact. It is testing whether you noticed **which** fact the scenario made relevant.

The mechanics

	VALUE
Questions	65 total. 50 scored, 15 unscored and not identified
Duration	130 minutes. Exactly 2 minutes per question
Score	Scaled 100 to 1,000. Pass at 720
Scoring model	Compensatory. No per-domain pass mark
Guessing	No penalty. Never leave a question blank
Domains	Development 32%, Security 26%, Deployment 24%, Troubleshooting 18%
ESL	An extra 30 minutes, requested before you schedule

ON THE EXAM

The 15 unscored questions are the reason a question that feels unreasonably obscure may simply not count. Answer it, and move on without letting it unsettle you.

What compensatory scoring means

- You need **720 overall**. There is no minimum in any individual domain.
- So a weak domain can be carried by strong ones. Nothing is a knockout.
- But the weights are not equal. **Development is 32%** and Troubleshooting is 18%.
- Improving your weakest large domain is worth more than perfecting a small one.
- The scaled score is **not a percentage**. 720 does not mean 72% of questions correct.

> ON THE EXAM
>
> The last line matters for your expectations. Candidates commonly report needing roughly 70 to 75% correct, but the mapping is not published and varies by exam form.

The two question types

- **Multiple choice:** one correct answer, three distractors. The majority of the exam.
- **Multiple response:** two or more correct, usually from five options.
- The question **states how many to choose**. Choose TWO means exactly two.
- **There is no partial credit.** Two of three correct scores zero, exactly as zero of three would.
- So on a multiple response question, treat each option as its own true-or-false decision.

> ON THE EXAM
>
> The no-partial-credit rule changes the strategy. On a Choose TWO you need both right, so it is worth more time than a single-answer question, not less.

The anatomy of a question

- **The setting:** what the system is and does. Often the longest part, and often the least useful.
- **The constraints:** what must be true. This is where the deciding clause hides.
- **The ask:** the actual question, usually one sentence at the end.
- **The qualifier:** most cost-effective, least operational overhead, and so on.
- The setting is padding as often as it is signal. The constraints rarely are.

> ON THE EXAM
>
> Recognising these four parts is what lets you read a long scenario quickly: you are scanning for constraints and the qualifier, not absorbing the whole story.

Reading order

- **Read the ask first**, at the bottom. It tells you what to look for.
- **Then read the scenario**, now knowing which details matter.
- **Then form an answer** before looking at the options.
- Only then read the options, checking whether your answer is among them.
- Reading options first anchors you. Every one will then look partly plausible.

> READ THIS TWICE
>
> Forming an answer before reading the options is the single highest-value habit here. It converts four plausible options into one match and three rejections.

The qualifiers, and what each selects

THE QUALIFIER	IT SELECTS
MOST cost-effective	The cheapest option that still meets every stated requirement
LEAST operational overhead	The most managed option, even where it costs more
MINIMAL code change	Configuration over rewriting, however elegant the rewrite
MOST secure	Least privilege, no long-lived credentials, encryption by default
MOST highly available	Multi-AZ or multi-Region, accepting the cost
MOST likely cause	Diagnosis. The other options may be possible but not probable
BEST practice	The AWS-idiomatic answer, not the one you would build

ON THE EXAM

Worth screenshotting. When two options both work, the qualifier is the tiebreaker, and reading past it is how you choose a defensible answer that is not the intended one.

The deciding clause

- Most scenarios contain a sentence that **eliminates all but one option**.
- It is often short, and rarely in the first paragraph.
- Examples: cannot modify the code, must not reduce capacity, users must not reset passwords.
- A negative statement is frequently the clue. **Versioning is not enabled** exists to rule an option out.
- If you cannot name the clause, you have probably not found the real constraint.

> ON THE EXAM
>
> This course used that construction deliberately in every module. When a scenario states something apparently incidental, ask which option it was written to eliminate.

How this course was built

- Every worked scenario in this course used the same construction deliberately.
- A setting, constraints, an ask, and one clause that eliminates the strongest distractor.
- Each solution named that clause explicitly, under **The signal**.
- Several modules built a **near miss** option: correct engineering, wrong for the scenario.
- That was practice for this, not padding. The habit is what transfers.

> ON THE EXAM
>
> If you have been naming the deciding clause before checking the answer, you have already been doing the thing this module describes.

The six kinds of wrong answer

TYPE	WHAT IT LOOKS LIKE
Right service, wrong feature	DAX for an RDS cache, or a GSI to spread writes
Right for a different scenario	A perfectly good answer to a question that was not asked
Solves it by dropping a requirement	Make the bucket public, or remove the audit trail
Works, but ignores the qualifier	Correct engineering, at ten times the cost or effort
Belongs to another platform	ValidateService on Lambda, or active tracing on ECS
Formerly correct	The pre-2020 answer, still plausible if you learned it once

ON THE EXAM

The fourth type is the most dangerous, because nothing about it is factually wrong. It loses only against the qualifier you were meant to read.

The three easy eliminations

- **Wrong platform.** An EC2-only hook in a Lambda question is gone immediately.
- **Drops a requirement.** Making a bucket public, or abandoning the audit trail.
- **Rebuilds a managed feature by hand.** A Lambda function counting what a metric filter counts.
- These three take seconds and typically remove one or two options.
- That converts a four-way guess into a two-way decision, which is worth real marks.

> ON THE EXAM
>
> Elimination is not a substitute for knowing the answer, and it is what rescues the questions where you do not. Two options at 50% beats four at 25%.

When two options both work

- This is normal, not a sign you have misread. Distractors are meant to be defensible.
- **Check the qualifier first.** It exists precisely for this case.
- **Then look for a constraint** that one option violates and the other does not.
- Prefer the **managed** option, the one with less code, and the one AWS documents.
- Prefer the simpler option. A complex answer is rarely the intended one at associate level.

> ON THE EXAM
>
> The AWS-idiomatic tiebreaker is genuinely predictive: managed over self-built, configuration over code, loosely coupled over integrated.

Weak signals worth knowing

- Options containing **always** or **never** are more often wrong than right.
- An option naming a service that appears nowhere in the scenario is often filler.
- **The longest option is not reliably correct.** That heuristic does not survive contact with AWS.
- Neither is any pattern in the letter. Answer positions are randomised.
- Use these only to break a genuine tie, never in place of reasoning.

> ON THE EXAM
>
> Stated honestly because the internet repeats the length heuristic constantly. It is not reliable, and building a habit on it will cost you marks.

Time management

- **130 minutes, 65 questions, exactly 2 minutes each.** That is the whole budget.
- **At 2.5 minutes, flag it and move on.** A long question is worth the same as a short one.
- Check the clock at question 20 and question 40, not constantly.
- Aim to reach the end with 15 to 20 minutes left for flagged questions.
- Running out of time with questions unanswered is the avoidable way to fail.

> ON THE EXAM
>
> Every question is worth the same. Spending six minutes on a hard one costs you two easy ones you never reached, which is a net loss of marks.

Flagging and reviewing

- **Always answer before flagging.** A flagged blank becomes a blank if time runs out.
- Flag two kinds: ones you could not decide, and ones you rushed.
- Later questions sometimes remind you of a fact an earlier one needed.
- **Change an answer only for a reason you can name.** Not because it feels wrong now.
- Reviewing everything is not possible in the time. Review the flags.

> ON THE EXAM
>
> The named-reason rule matters. Second-guessing on a feeling changes correct answers to incorrect ones more often than the reverse.

When you genuinely do not know

- **Eliminate the wrong platform and the requirement-dropping options.** Usually one or two go.
- **Apply the qualifier** to whatever remains.
- Prefer managed over self-built, and configuration over code.
- Pick one, flag it, move on. Do not spend a third minute.
- It may be one of the 15 unscored questions. You will never know which.

> ON THE EXAM
>
> That last point is worth internalising before exam day. An unreasonably obscure question may simply not count, so it deserves no more than its two minutes.

What the exam does not do

- **It does not require memorised pricing.** Relative cost, yes. Dollar figures, no.
- It does not require writing code. Snippets appear, usually Python with boto3.
- It does not test CLI flags or console click paths.
- It does not use trick questions. Ambiguity is a badly written question, not a trap.
- It does expect **limits and defaults**: 15 minutes, 29 seconds, 400 KB, 5 GB.

> ON THE EXAM
>
> The last line is where revision effort pays. Numbers decide questions outright, and there are perhaps thirty that matter across the whole exam.

These four are different: each shows you a question and asks how it works.

A question describes a team running scheduled reports on EC2, and asks for the MOST cost-effective way to run them, given they take 40 minutes and run twice a day. Every option would technically work.

One option is a Fargate task on a schedule. One is a Lambda function. One is a persistent EC2 instance. One is an EC2 instance started and stopped by a script.

What is the fastest correct way to resolve this question?

A Compare the hourly rates of Fargate and EC2 and pick the cheaper

B Eliminate Lambda on the 15-minute limit, then apply the qualifier to what remains

C Choose the longest option, since detailed options are usually correct

D Choose the persistent EC2 instance, as it has the lowest per-hour price

◆ PAUSE AND DECIDE

What is the fastest correct way to resolve this question?

A **Compare hourly rates**

The exam does not require memorised pricing, and it would not eliminate Lambda, which fails on a hard limit rather than on price.

B **Eliminate on the limit, then qualify**

40 minutes exceeds Lambda's 15-minute ceiling, so that option is gone on a fact. The qualifier then chooses between the remaining three on idle cost.

C **Longest option**

The length heuristic is unreliable and is not a reasoning method. It happens to be wrong here as often as right.

D **Persistent EC2**

The lowest hourly rate for something running 24 hours a day to do 80 minutes of work. Cheapest per hour is not cheapest overall.

THE SIGNAL

Two steps in order: a hard limit eliminates, then the qualifier chooses. The stated duration is doing the first job, and 'MOST cost-effective' the second.

> A question asks how to give a third-party auditor read access to CloudTrail logs in an S3 bucket. The requirements state the bucket must not be publicly accessible and access must be revocable.
>
> One option reads: Enable public read access on the bucket and share the URL with the auditor.

What kind of distractor is that option?

A Right service, wrong feature

B Formerly correct, now outdated

C Works, but ignores the qualifier

D Solves the problem by dropping a stated requirement

◆ PAUSE AND DECIDE

What kind of distractor is that option?

A Right service, wrong feature

That type uses a real feature of the right service incorrectly, like DAX in front of RDS. Public access is not a misused feature here, it is a violated constraint.

B Formerly correct

Public buckets were never the documented way to share audit logs. This was not good practice at any point.

C Ignores the qualifier

That type is technically valid but loses on cost or overhead. This option fails a hard requirement, not a preference.

D Drops a stated requirement

The scenario says the bucket must not be publicly accessible. The option achieves access by removing that requirement, which is never permitted.

THE SIGNAL

Naming the type is faster than evaluating the option. Requirement-dropping distractors can be eliminated the moment you match them against the constraints.

A question describes a Lambda function that must read a database credential rotating every 30 days. Two options remain after elimination: store it in Secrets Manager and fetch it per invocation, or store it in Secrets Manager and fetch it during initialisation.

The question asks for the approach with the LOWEST latency. The scenario does not mention rotation timing beyond the 30-day period.

How should this be resolved?

A Fetch during initialisation. The qualifier is latency, and a 30-day rotation tolerates a warm environment's lifetime

B Fetch per invocation, because correctness always outranks latency

C Neither. The question is ambiguous and should be flagged and skipped

D Fetch per invocation, since it is the option that appears more often in AWS documentation

◆ PAUSE AND DECIDE

How should this be resolved?

A Initialisation, following the qualifier

The qualifier is LOWEST latency, and nothing in the scenario makes staleness a stated constraint. A 30-day rotation comfortably exceeds any environment's lifetime.

B Per invocation, correctness first

Imports a priority the question did not state. When the qualifier names latency, latency is the tiebreaker.

C Flag and skip

The question is not ambiguous once the qualifier is read. Skipping a resolvable question costs a mark you could have had.

D Documentation frequency

Not a reasoning method, and not true. Fetching at initialisation is the documented pattern for exactly this case.

THE SIGNAL

Module 19's scenario 4 had the opposite answer, because it stated a 7-day rotation and required rotated credentials to be picked up. The qualifier and the constraints changed, so the answer did.

> You are 95 minutes into the exam and have reached question 48. Eleven questions are flagged. Question 48 is a Choose TWO about DynamoDB indexes, and you are confident about one option and torn between two others.
>
> You have 35 minutes remaining.

What is the best use of the remaining time?

A Spend as long as needed on question 48, since Choose TWO questions are worth more

B Skip question 48 entirely and use the time on the eleven flagged questions

C Answer 48 with your best pair, flag it, finish the remaining questions, then review flags

D Change your earlier flagged answers to whichever now feels more likely

◆ PAUSE AND DECIDE

What is the best use of the remaining time?

A. Take as long as needed

A Choose TWO is worth the same as any other question. 35 minutes for 18 questions is already under two minutes each, so overspending here loses easy marks later.

B. Skip it entirely

Leaves a blank on a question you partly know, and there is no penalty for guessing. Never leave anything unanswered.

C. Answer, flag, finish, then review

Records a scored answer, preserves the option to reconsider, and prioritises reaching the 17 unseen questions over perfecting one.

D. Revise flagged answers on feeling

Changing without a nameable reason turns correct answers into incorrect ones more often than the reverse. It also abandons unseen questions.

THE SIGNAL

Unseen questions outrank flagged ones. A question you have not read is a guaranteed zero if you never reach it; a flagged one already has an answer recorded.

What to carry forward

- **Read the ask first, form an answer, then read the options.** The highest-value habit.
- **Find the deciding clause.** If you cannot name it, you have not found the constraint.
- **The qualifier is the tiebreaker** when two options both work. It is not decoration.
- Eliminate wrong platform, requirement-dropping and hand-built options first. It is fast.
- **Two minutes each.** Flag at 2.5, always answer before flagging, never leave a blank.

> ON THE EXAM
>
> Next: the revision gap, which gathers every currency note in this course into one place. Then three practice exams.